Lecture 11

19 October 2026 23:53

After building the feed forward class, now we will make the block class

The next step is where this truly becomes a **Deep Transformer**!

Right now, our model only has **1 round** of communication (sa_heads) and **1 round** of thinking (ffwd).

To make the model really smart, we want to **stack multiple layers** on top of each other:
$Tokens \longrightarrow Block\ \ 1 \longrightarrow Block\ \ 2 \longrightarrow Block\ \ 3 \longrightarrow \cdots \longrightarrow Block\ \ N \longrightarrow Logits$

The Big Problem with Deep Networks

If you just stack 4 or 6 layers of attention and feed-forward, the network becomes too deep to train:

V IMP
1: Gradients shrink as they flow backwards (the **vanishing gradient problem**). ----> i.e. while reducing the loss, we have to backprapogate and this will loss data to make correction, i.e. the weights work becomes more hard, we can see this as loosing efficiency

2. Training slows to a crawl or stalls completely.

To solve this, the Transformer uses **two life-saving techniques**:

1. Residual Connections (Skip Connections)

Instead of forcing information through the heavy layers, we add a **direct highway** that bypasses the layer:

$Output = x + Layer\,(x)$

```
x (input)
|─┐
|  ▼
| [Attention Layer]
|   /
▼  ▼
x + Attention(x)  <-- Added together!
```

- **Why?** During backpropagation, addition distributes gradients equally.
- The gradient can flow straight down this "highway" from the loss all the way back to the input embeddings without getting destroyed!

2. Layer Normalization (nn.LayerNorm)

In deep networks, activations can slowly drift and become too large or too small.

- **nn.LayerNorm** normalizes the 32 features of each token so they have **mean = 0** and **variance = 1**.
- In modern Transformers (like GPT-2 and GPT-3), we apply LayerNorm **before** the layer (called *Pre-LayerNorm*):

$x = x + Attention\left(LayerNorm_1\,(x)\right)\ x = x + FeedForward\left(LayerNorm_2\,(x)\right)$

3. The Block Class

We bundle one round of Attention + one round of FeedForward + Residuals + LayerNorm into a single, reusable **Block**:

Python

```
class Block(nn.Module):
    """ Transformer block: communication followed by computation """

    def __init__(self, n_embd, n_head):

        # n_embd: embedding dimension, n_head: the number of heads we'd like
        super().__init__()

        head_size = n_embd // n_head

        self.sa = MultiHeadAttention(n_head, head_size)

        self.ffwd = FeedForward(n_embd)

        self.ln1 = nn.LayerNorm(n_embd)

        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        # Notice the residual additions (x + ...) and LayerNorms:

        x = x + self.sa(self.ln1(x))    # Communicate with residual highway
        x = x + self.ffwd(self.ln2(x))  # Think with residual highway
        return x
```

Look how clean the main model becomes!

In your BigramLanguageModel, instead of manually calling sa_heads and ffwd, you just create a stack of blocks:

python
1. In __init__:
```
self.blocks = nn.Sequential(
    Block(n_embd, n_head=4),
    Block(n_embd, n_head=4),
    Block(n_embd, n_head=4),
    nn.LayerNorm(n_embd), # final layer norm before predicting
)
```

2. In forward:
```
x = tok_emb + pos_emb   # (B, T, C)
x = self.blocks(x)      # (B, T, C) passes through all blocks!
logits = self.lm_head(x)# (B, T, vocab_size)
```

But why do we need so many layers, we are taking many random batches to train the model

Understand it by this :

Question ask to antigravity :
you are telling that, our model is having only one round each for communcation adn feedforward, but we are taking random many batch and doing these process, so why do we need more.

1. The confusion comes from mixing up **Practice over time** with **Thinking depth on a single problem**.

1. Batches / Iterations = "Practice over time"

When we run 5000 iterations with random batches:

- That is like a student doing **5,000 practice problems** from a textbook.
- Each iteration shows the model a new piece of text so it can learn from its mistakes.

BUT: No matter how many practice problems the student does, look at how the student's brain works on **any single problem**:

2. Layers / Blocks = "Steps of reasoning on a single sentence"

Right now, when our model reads a sentence, it only gets **1 round**:

1. Characters exchange info **once** (sa_heads).
2. Each character reflects **once** (ffwd).
3. **BAM — immediately guess the next character!**

Think about how shallow that is. In only 1 round:

- Token A can only talk to Token B directly.
- Token A cannot understand complex, multi-step ideas.

What happens when you add more rounds (Layers)?

In deep learning, layers build **hierarchies of understanding**:

Layer 1 (Round 1): Characters talk to characters
└— Model learns: "t" and "h" often form "th". Basic syllables.

Layer 2 (Round 2): Syllables talk to syllables
└— Model learns: Words! ("king", "lord", "shall").

Layer 3 (Round 3): Words talk to words
└— Model learns: Grammar & syntax. ("The king [verb]...")

Layer 4 (Round 4): Phrases talk to phrases
└— Model learns: Context & tone. ("This is a dramatic royal speech").

The Detective Analogy

Imagine 4 detectives trying to solve a murder:

- **1 Layer (1 Round):**
  - Detectives meet for **5 minutes**, share initial clues, think for 2 minutes, and must immediately name the killer.
  - Even if they solve 5,000 cases over their career (**5,000 iterations**), on *any individual case*, their investigation is rushed and shallow!

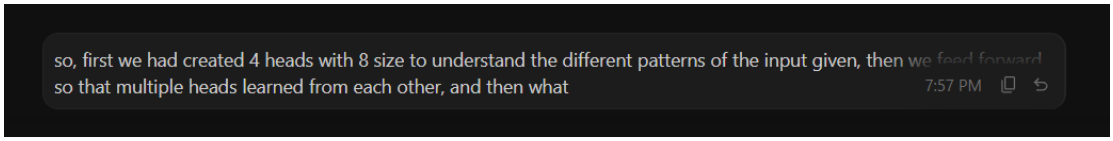
- **Multi-Layer (Multiple Rounds):**
  - **Round 1:** Share raw clues.
  - **Round 2:** Re-talk based on what everyone learned in Round 1:
    "Wait! Alice noticed the door was locked, and Bob found muddy footprints. Let's connect those two!"
  - **Round 3:** Synthesize everything into the complete story.

The Key Takeaway:

- **Iterations (5000):** How many times the model updates its weights.

- **Layers (Blocks):** How many steps of logical deduction the model can perform **on a single sentence** before making a prediction.
-

That's why real GPT models don't just have 1 layer—they have **12, 24, or even 96 layers** stacked on top of each other!



Here is what happens **next** to complete the entire Transformer:

Step 1: Wrap it with the "Safety Highway" (Residuals & LayerNorm)

Right after Attention and FeedForward, we don't just throw away the input x. We **add it back** and normalize it:

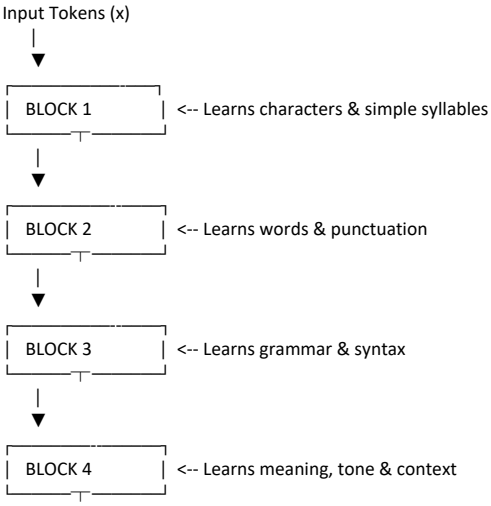
$x = x + Attention\left(LayerNorm_1(x)\right)$ $x = x + FeedForward\left(LayerNorm_2(x)\right)$

- **Why + x (Residual)?** It ensures the token never forgets its original identity while adding new insights from other tokens. It also creates a smooth highway for gradients during backpropagation.
- **Why LayerNorm?** It keeps the numbers stable (mean 0, variance 1) so they don't blow up as numbers multiply.

This complete package is called **1 Block**.

Step 2: Repeat! (Stack Multiple Blocks in a Row) 🧱

One block gives you 1 round of discussion and 1 round of thinking.
"And then what?" We pass the result into **another Block**, and then **another Block**!



Each block takes the 32 features, refines them, and hands them to the next block to think even deeper.

Step 3: The Finish Line (lm_head) 🏁

After the token vectors have traveled through all 4 blocks, they are now packed with deep contextual knowledge.
Now comes the final prediction:

1. **Final LayerNorm:** A final polish on the features:

   python
   x = self.ln_f(x) # (B, T, 32)
2. **lm_head (The Judge):** Projects the 32 refined features into the 65 possible characters:

   python
   logits = self.lm_head(x) # (B, T, 65)
3. **Softmax & Sample:** Turns the 65 logits into probabilities and picks the next character!

The Grand Picture:

[Equation]

Change 1 :

Step 1: Stacking Simple Blocks (Where you are right now)

Python

class Block(nn.Module):

```
def __init__(self, n_embd, n_head):
    super().__init__()
    head_size = n_embd // n_head
    self.sa = MultiHeadAttention(n_head, head_size)
    self.ffwd = FeedForward(n_embd)

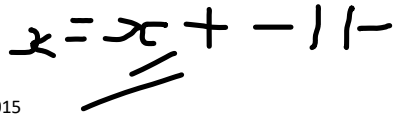
def forward(self, x):
    x = self.sa(x)      # communicate
    x = self.ffwd(x)  # think
    return x
```

Inside BigramLanguageModel:

python
```
self.blocks = nn.Sequential(
    Block(n_embd, n_head=4),
    Block(n_embd, n_head=4),
    Block(n_embd, n_head=4),
)
```

- **Why this change?**
  To give the network **3 consecutive rounds** of communication and thinking, building up from letters → words → grammar.
- **The Problem that appears:**
  As neural networks get deeper, the **gradients struggle to backpropagate** through so many layers of matrix multiplications. The loss stops improving or trains very poorly.

Step 2: Adding Residual Connections (Skip Connections)



To fix the vanishing gradient problem, Karpathy introduces **Residual Connections** (from the famous 2015 ResNet paper):

python
```
def forward(self, x):
    x = x + self.sa(x)    # <-- ADD "+ x" (Skip connection)
    x = x + self.ffwd(x)  # <-- ADD "+ x" (Skip connection)
    return x
```

- **Why is + x needed?**

In calculus, addition distributes gradients equally: $\frac{\partial}{\partial x}(x + f(x)) = 1 + f'(x)$ That "1" is a pure **gradient superhighway**! Gradients from the loss can flow straight back down the highway all the way to the input embeddings without getting shrunk by layer weights.

V IMP :
The companion tweak: Projection layer (self.proj)
Inside MultiHeadAttention, Karpathy adds a projection layer right after concatenating heads:

python
```
self.proj = nn.Linear(n_embd, n_embd)
```

- **Why?** It projects the concatenated heads back into the residual pathway so the dimensions and features blend smoothly back into x.

If everything seems complex this is the easy summary :

Everything in this architecture has **one golden rule**:

> **Every token enters as a vector of size 32 (n_embd), and must leave as a vector of size 32 (n_embd).**

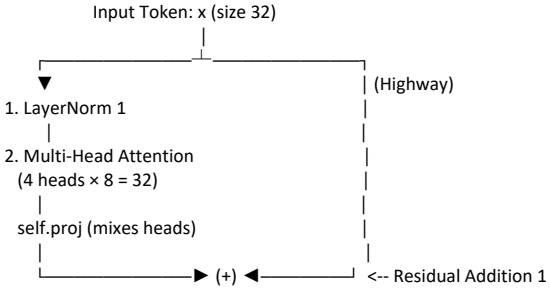
Why? Because they all travel down the same **highway (x)**.

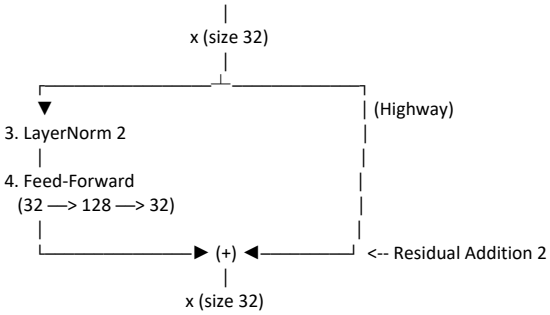


Highway is the path made so that the final x will be passed through different blocks but there are 4 block made which does not relayed with 4 heads, its only from the 'attention is all you need ' paper

🧱 The 5 New Pieces and How They Connect

V Imp ----diagram for understanding ;





📝 Notes to Copy: Piece-by-Piece Breakdown

1. MultiHeadAttention (Communication)

1. **IMP : What it does:** Creates 4 smaller heads (size 8 each). Each head looks for a different pattern (verbs, grammar, quotes, rhymes).
2. **Why it's needed:** A single head only sees one relationship. 4 heads see 4 relationships in parallel.
2. **IMP : Shape change:** $32 \longrightarrow (4 \times 8) \longrightarrow 32$ (via torch.cat).

2. self.proj inside MultiHeadAttention (The Mixer)

1. **What it does:** A linear layer: nn.Linear(32, 32).
2. **IMP : Why it's needed:** When the 4 heads are concatenated, their outputs are just glued side-by-side. self.proj **mixes the 4 heads together** so their discoveries blend into one unified 32-dim thought.
3. **IMP : Connection to others:** Ensures the attention output is exactly size 32 so it fits the residual highway.

3. Residual Connections: x = x + ... (The Highway)

1. **V IMP : What it does:** Instead of replacing x with the layer's output, it **adds** the output back to x: $x = x + Attention(x)$
   $x = x + FeedForward(x)$
2. **ALSO V IMP : Why it's needed:** As models get deep, gradients shrink to zero during backpropagation (vanishing gradients). Addition creates an uninterrupted **superhighway** for gradients to flow backward straight to the input!
3. **Key point to remember : Connection to others:** This is the reason **everything must output size 32**—you cannot add tensors with different sizes!

4. FeedForward (Thinking Individually)

1. **What it does:** An MLP applied to each token: $nn.Linear(32, 128) \longrightarrow ReLU() \longrightarrow nn.Linear(128, 32)$
2. **V IMP : Why it's needed:** Attention only *collected* clues from other tokens. Now, the token needs to *think* and combine those clues to form higher-level meaning.
3. **Why expand to 128?** Gives the network temporary "scratchpad room" to evaluate complex non-linear combinations before compressing back to 32.
4. **Connection to others:** Takes the mixed output from Attention, processes it, and outputs size 32 to add back to the highway.

5. LayerNorm (The Balancer)

1. **What it does:** Normalizes the 32 numbers of each token so their **mean is 0** and **variance is 1**.
2. **V IMP : Why it's needed:** Because of all the additions (x = x + ...), numbers can slowly drift, grow too big, or explode. LayerNorm calms the numbers down.
3. **Connection to others:** Applied right before Attention (ln1) and right before FeedForward (ln2) to keep inputs stable and healthy.

We have saw this before:
The non-linearity -->
weught of 1st is added to the next weught so th learns faster

🎯 The 1-Sentence Summary for You:

Attention gathers the clues from other tokens, proj blends those clues together, LayerNorm keeps numbers stable, FeedForward thinks about what was learned, and the + x residual highway makes sure nothing is forgotten and gradients can flow!

After commenting out the multi-head attention and the ffwd line only print this :
x = self.block(x)

And the loss has down to 2.000 but the val loss is little bit more then, the train loss